

**Sofia Lima dos Santos**  
**Paradigmas da Programação - Lista 2**

**Exercício 1**

```
#include <stdio.h>

int main(){
    int n;
    int fatorial = 1;

    printf("Digite um número: \n");
    scanf("%d", &n);

    if (n < 0){
        printf("Inválido. Digite um número positivo. \n");
    } else {
        for(int i = 1; i <= n; i++) {
            fatorial = fatorial * i;
        }
        printf("O fatorial é: %d\n", fatorial);
    }

    return 0;
}
```

**Exercício 2**

```
#include <stdio.h>

int calculaFatorial(int n){
    if (n == 0){
        return 1;
    } else{
        return n * calculaFatorial(n - 1);
    }
}

int main(){
    int n;
```

```

printf("Digite um número: \n");
scanf("%d", &n);

if (n < 0){
    printf("Inválido. Digite um número positivo. \n");

} else {
    int resultado = calculaFatorial(n);
    printf("O fatorial de %d é: %d\n", n, resultado);
}

return 0;
}

```

### Exercício 3

```

#include <stdio.h>

int fibonacci(int n) {
    int t1 = 0, t2 = 1, proximo;
    if (n == 0) {
        return t1;
    }
    for (int i = 2; i <= n; ++i) {
        proximo = t1 + t2;
        t1 = t2;
        t2 = proximo;
    }
    return t2;
}

int main() {
    int n;
    printf("Digite um número: ");
    scanf("%d", &n);

    printf("O termo %d da sequência de Fibonacci é: %d\n", n,
    fibonacci(n));

    return 0;
}

```

#### Exercício 4

```
#include <stdio.h>

int fibonacci(int n) {
    if (n <= 1) {
        return n;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int n;
    printf("Digite um número: ");
    scanf("%d", &n);

    printf("O termo %d da sequência de Fibonacci é: %d\n", n,
    fibonacci(n));

    return 0;
}
```

#### Exercício 5

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int verificaPrimo(int n){
    if(n < 2){
        return 0;
    }

    for (int i = 2; i <= n/2; ++i){
        if (n % i == 0) {
            return 0;
        }
    }
    return 1;
}

int main(){
    int n_gerados = 0;
```

```

int c = 0;
srand(time(NULL));

while (c < 25){
    int n = rand() % 100;

    if (verificaPrimo(n)){
        c++;
    }
    n_gerados++;
}

printf("Quantidade de números gerados até encontrar 25 números
primos: %d\n", n_gerados);

return 0;
}

```

## Exercício 6

```

#include <stdio.h>

int main(){

int n1, n2, soma1 = 0, soma2 = 0;

    printf("Digite o primeiro número: \n");
    scanf("%d", &n1);

    printf("Digite o segundo número: \n");
    scanf("%d", &n2);

    for(int i = 1; i < n1; ++i){
        if (n1 % i == 0){
            soma1 += i;
        }
    }

    for(int i = 1; i < n2; ++i){
        if (n2 % i == 0){
            soma2 += i;
        }
    }
}

```

```

    if (soma1 == n2 && soma2 == n1){
        printf("%d e %d são números amigos! \n", n1, n2);
    } else{
        printf("%d e %d não são números amigos! \n", n1, n2);
    }

    return 0;
}

```

## Exercício 7

```

#include <stdio.h>

int main(){
    int n, soma = 0;

    printf("Digite um número: \n");
    scanf("%d", &n);

    for(int i = 1; i < n; ++i){
        if (n % i == 0){
            soma += i;
        }
    }

    if (soma == n){
        printf("%d é um número perfeito! \n", n);
    } else{
        printf("%d não é um número perfeito! \n", n);
    }

    return 0;
}

```

## Exercício 8

```

#include <stdio.h>

int mult(int a, int b) {
    if (b == 0) {
        return 0;
    }
}

```

```

    return a + mult(a, b - 1);
}

int main() {
    int a = 5;
    int b = 15;
    int r = mult(a, b);
    printf("%d", r);
    return 0;
}

```

### Exercício 9

```

#include <stdio.h>

int contaDigito(int n) {
    if (n == 0) {
        return 0;
    }
    return 1 + contaDigito(n / 10);
}

int main() {
    int n;
    int q;

    printf("Digite um número inteiro: \n");
    scanf("%d", &n);

    if (n == 0) {
        q = 1;
    } else {
        q = contaDigito(n);
    }

    printf("O número %d tem %d dígito(s)\n", n, q);
    return 0;
}

```

### Exercício 10

```

#include <stdio.h>

```

```

int array[] = {1, 70, 95, 100};
int tamanho = sizeof(array) / sizeof(array[0]);
int n;

int pertence(int n){
    for (int i = 0; i < tamanho; ++i) {
        if (array[i] == n) {
            return 1;
        }
    }
    return 0;
}

int main(){
    int n;
    printf("Digite um número: ");
    scanf("%d", &n);

    if (pertence(n)){
        printf("%d pertence ao array!\n", n);
    } else {
        printf("%d não pertence ao array!\n", n);
    }

    return 0;
}

```

### Exercício 11

```

#include <stdio.h>

int verificaPrimo(int n){
    if(n < 2){
        return 0;
    }

    for (int i = 2; i <= n/2; ++i){
        if (n % i == 0) {
            return 0;
        }
    }
    return 1;
}

```

```

}

int main() {

    for (int i = 0; i < 100; i++){
        if (verificaPrimo(i)){
            printf("%d", i);
        }
    }

    return 0;
}

```

## Exercício 12

```

#include <stdio.h>
#include <string.h>

int main() {
    char cpf[12];
    char d_verificador[3];
    int soma = 0, soma2 = 0;
    int d1 = 0, d2 = 0;

    printf("Digite um cpf para ser verificado: ");
    scanf("%s", cpf);

    d_verificador[0] = cpf[9];
    d_verificador[1] = cpf[10];
    d_verificador[2] = '\0';

    cpf[9] = '\0';
    cpf[10] = '\0';

    for(int i = 0; i < 9; i++){
        soma += (cpf[8 - i] - '0') * (i + 2);
    }

    if((soma % 11) < 2){
        d1 = 0;
    } else {
        d1 = 11 - (soma % 11);
    }
}

```



```
}

cpf[9] = d1 + '0';

for(int i = 0; i < 10; i++){
    soma2 += (cpf[9 - i] - '0') * (i + 2);
}

if((soma2 % 11) < 2){
    d2 = 0;

} else {
    d2 = 11 - (soma2 % 11);
}

if (d_verificador[0] == (d1 + '0') && d_verificador[1] == (d2 + '0')) {
    printf("O CPF é válido\n");
} else {
    printf("O CPF não é válido\n");
}

return 0;
}
```